Convolutional Neural Networks

Content

Introduction

Convolution operation

Convolutional layers

Pooling layers

Other layers

Pre-trained models

Introduction

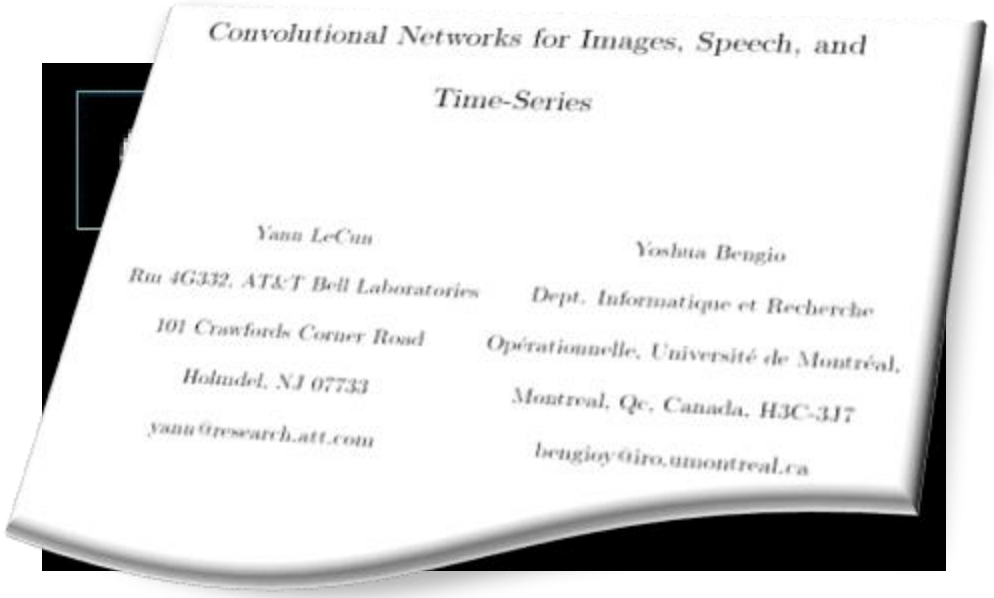
Convolutional Neural Networks (CNNs) are a type of neural networks initially designed to process and classify mainly images, speech and time series

- Fully connected (dense) layers are not invariant to translation

Translation and distortion invariance

- Locality
- Shared weights
- Dimensional (spatial/temporal) subsampling

Convolutions can be parallelly performed



LeCun, Y., Bengio, Y., & et al. (1995). Convolutional networks for images, speech, and time series. The Handbook of Brain Theory and Neural Networks (p. 3361). MIT Press.

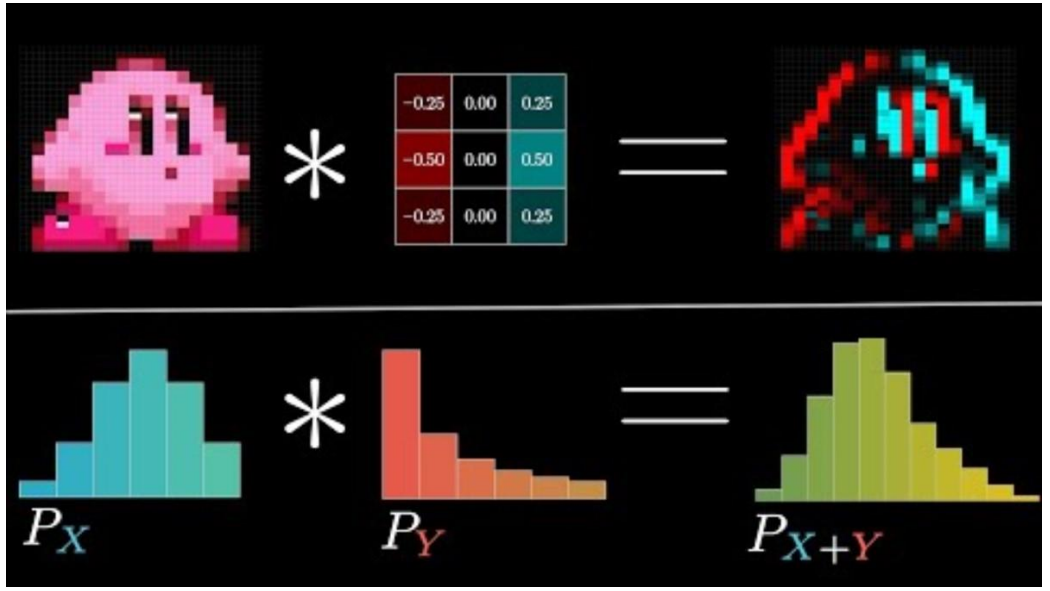
Convolution operation

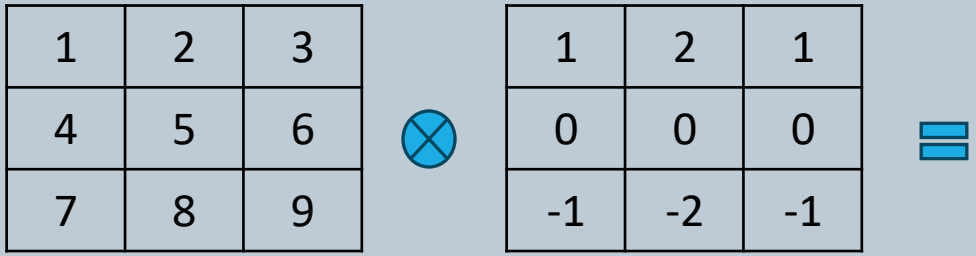
Measures the overlap between two functions when one is flipped and shifted

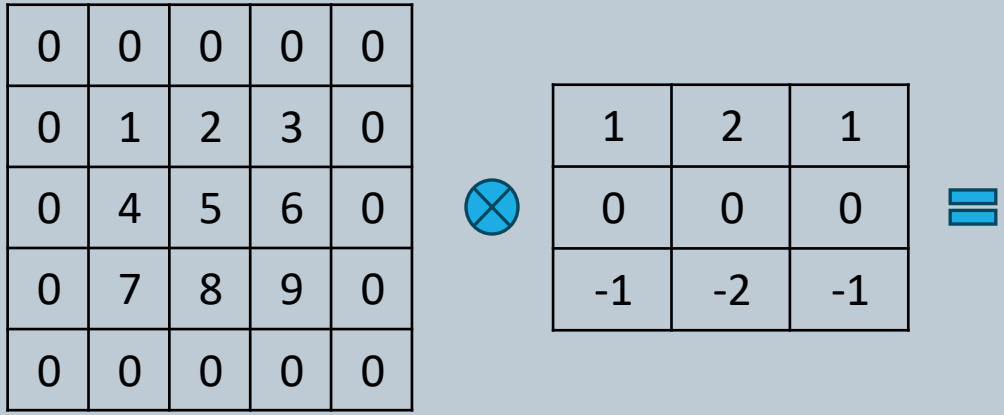
$$(f \cdot g)(x) = \int f(z)g(x-z)dz$$

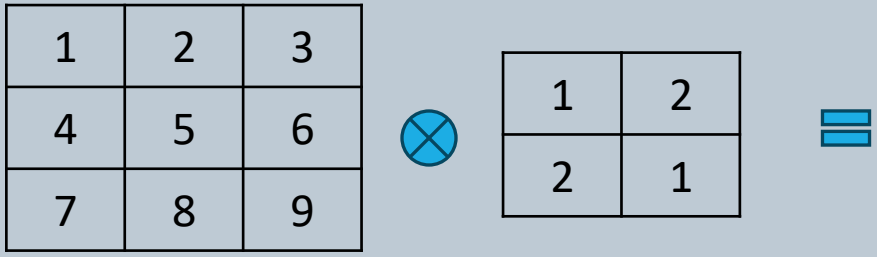
For 2D discrete space

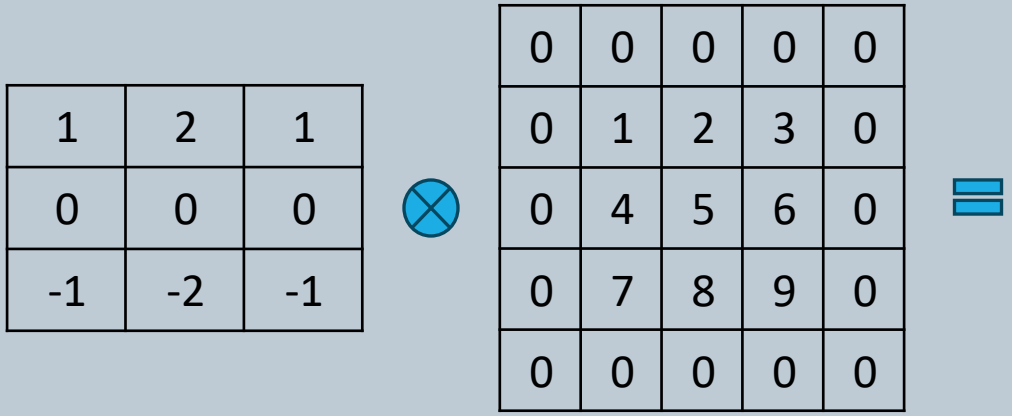
$$(f \cdot g)(x,y) = \sum_i \sum_j f(i,j)g(x-i, y-j)$$











Exercises

Calculate the above convolutions

Convolutional layers

$$output\ size = \left\lfloor \frac{input\ size - kernel\ size + 2 * padding}{stride} \right\rfloor + 1$$

Filter/kernel size

- Larger filters capture broader spatial context and more complex patterns, but produce smaller feature maps
- Smaller filters focus on fine-grained details and yield larger feature maps

Stride

- Determines how many positions the filter moves after each operation
- Increasing stride reduces the resulting feature map size, causing loss of spatial resolution
- Smaller stride preserves more detail and produces denser feature maps

Padding

- Adds extra values (commonly zeros) around the input before convolution
- Same padding ensures the same dimensions between the input and output
- Valid padding (no padding) produces a smaller output and can remove information at the borders

Pooling layers

Dimensionality reduction

- Reduce the spatial dimensions by applying a function to the feature map using a moving window

Translation invariance

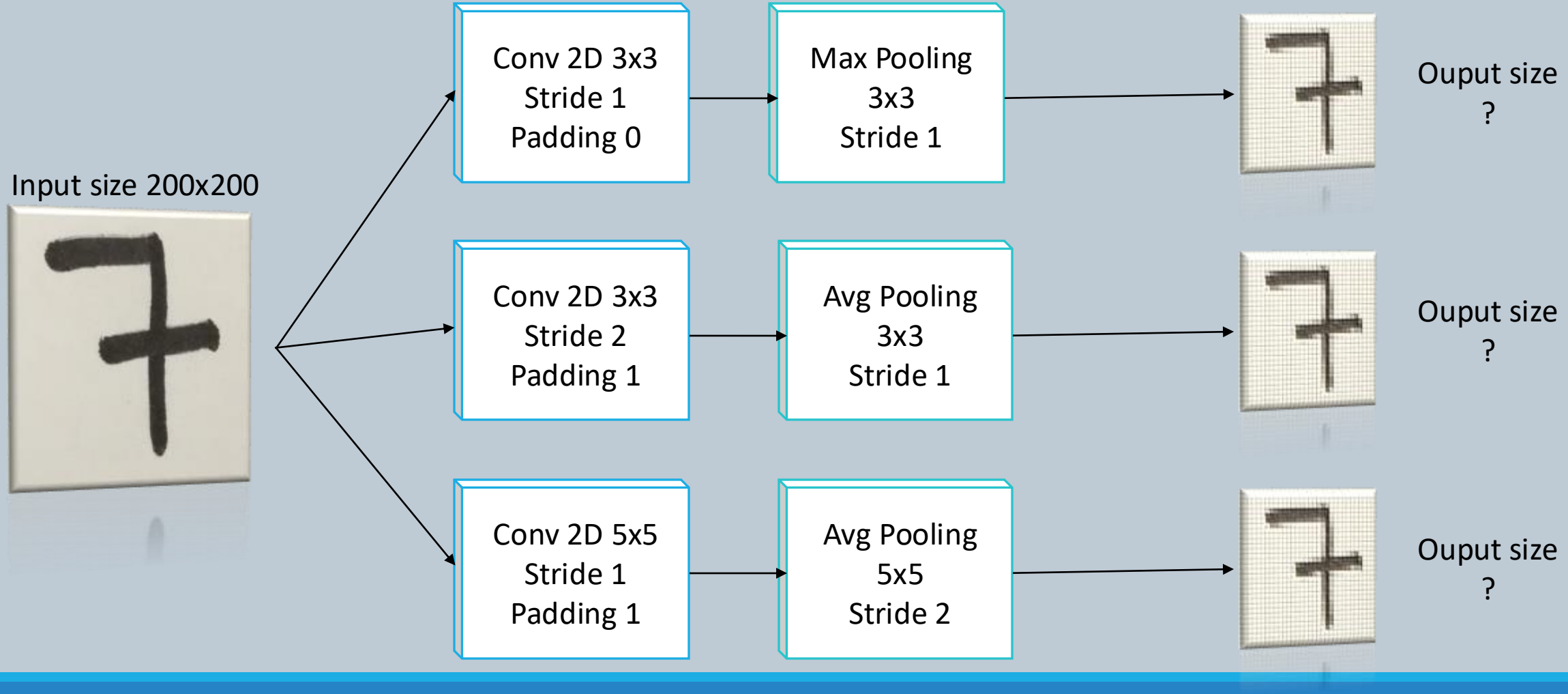
- Improve robustness to small shifts or distortions in the input

Features preservation

- Retain important features detected by convolutional filters while discarding less relevant details (noise)

Operations

- Window/filter size and stride define how the window moves over the feature map
- No learnable parameters
- Maximum pooling
- Average pooling = special case of a convolutional layer



Exercises

Calculate the ouput dimensions of the above operations

Other layers

Activation function (layer)

- Enables to capture non-linear features or complex spatial hierarchies
- Without activation function, it is equivalent to a single linear convolution

Batch normalization

- Normalizes the activations, accelerating training and stabilizing learning
- Typically placed after the convolution and before the activation function
- Reduces the need for bias (commonly disabled when using)

Dropout

- Randomly disables a fraction of neuron during training
- Prevent overfitting, forcing CNN learns redundant features, and improving generalization
- Reduce the potential correlation of spatial feature maps
- Lower rates for convolutional layers than for fully connected (dense) layers
- Usually placed after pooling layers



Lab Practice 3

Creating a simple CNN

Enrique Dominguez

DEEP LEARNING

Pre-trained models

LeNet

- Introduced by Yann LeCun in 1998 to recognize handwritten digits

AlexNet

- First modern CNN introduced by Alex Krizhevsky in 2012
- Winner of the ImageNet Large Scale Visual Recognition Challenge 2012

VGG

- Visual Geometry Group at Oxford University introduced the idea of using blocks in 2014
- VGG-11, VGG-16, VGG-19

GoogLeNet

- Winner of the ImageNet Challenge 2014
- Inception block consists of 4 parallel branches

ResNet

- Residual blocks include batch normalizations
- Introduced the adding operation of layers

DenseNet

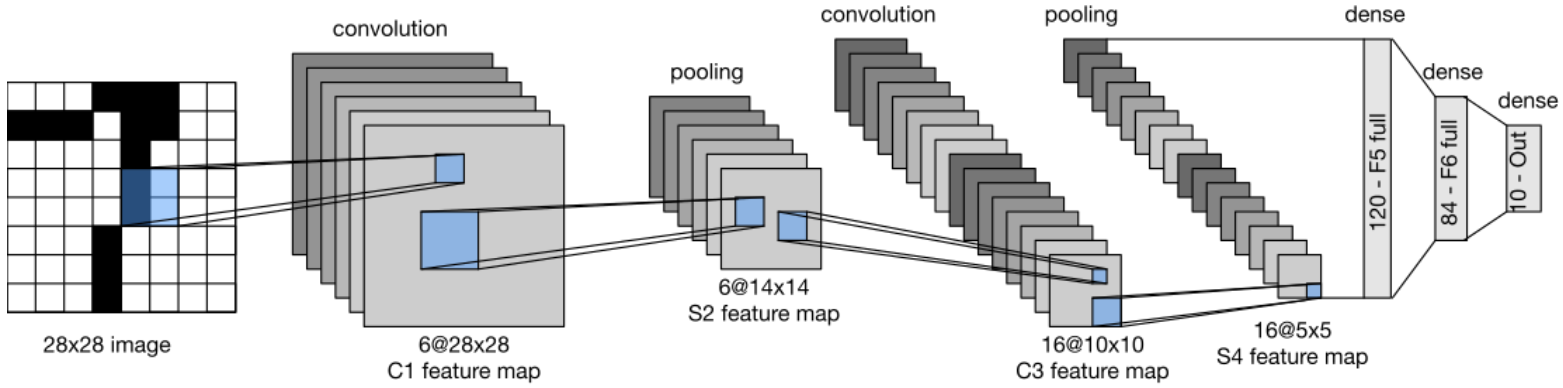
- Like ResNet, but using concatenation instead of addition

LeNet-5

One of the earliest CNN that demonstrated the feasibility of training networks by backpropagation on images

Architecture composed by 7 layers excluding the input

- 2 convolutional layers using sigmoid or tanh activations
  - 6 filters of size 5x5
  - 16 filters of size 10x10
- 2 average pooling using kernels of size 2x2 with stride 2
- 3 fully connected (dense) layers with 120, 84, and 10 neurons
- Input size of 32x32 images



AlexNet

Winner of the 2012 ImageNet challenge revolutionized computer vision

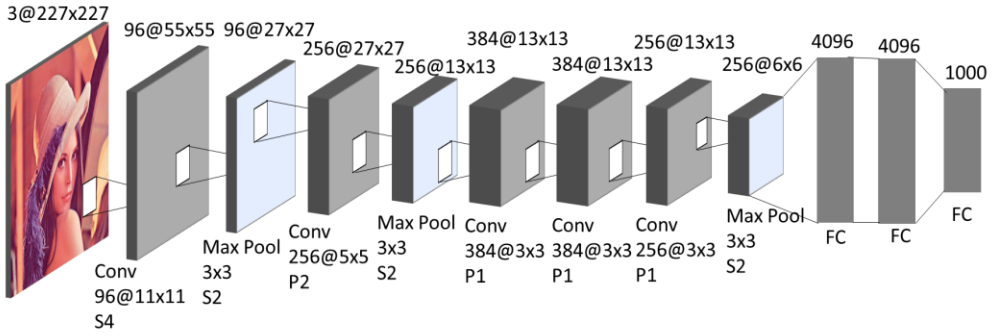
Highly influential, setting new standards and inspiring subsequent architectures

Architecture mainly composed by 5 convolutional layers and 3 fully connected layers

- Introduced ReLU activation improving training speed
- Overlapping Max Pooling to reduce spatial dimensions
- Uses Local Response Normalization (LRN) after early convolutional layers to help generalization
- Integrated dropout in fully connected layers to prevent overfitting
- Uses large filters in early convolutional layers and a high number of channels

Originally was trained in a dual GPU system

- Data augmentation
- Weight decay and SGD with momentum



VGG

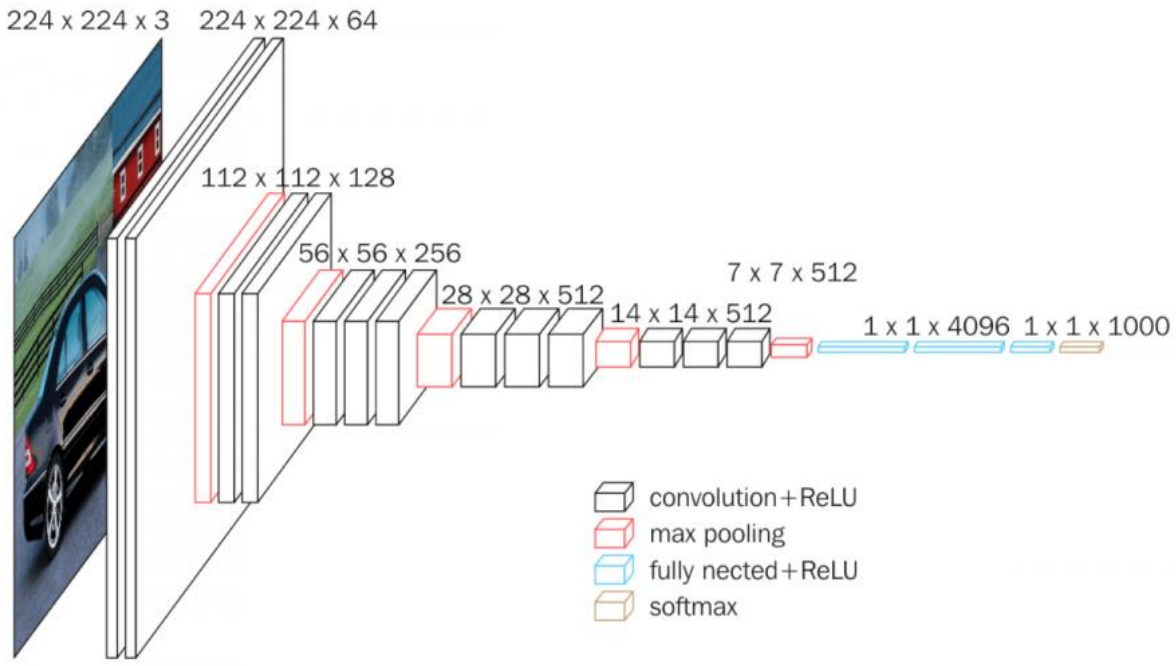
Deep network with a modular design and strong image classification performance

Popular pre-trained model for transfer learning

Deep and uniform architecture

- Mostly convolutional layers (16-19) and 3 fully connected layers
- Small filters (3x3) preserving spatial resolution to extract fine-grained features
- ReLU activation and Max Pooling for efficient training

Demonstrated that increasing network depth with uniform small filters improves performance



GooLeNet (Inception)

Inception module

- Multi-branch feature extraction using filters of size 1x1, 3x3, 5x5
- 1x1 convolutions for dimensionality reduction

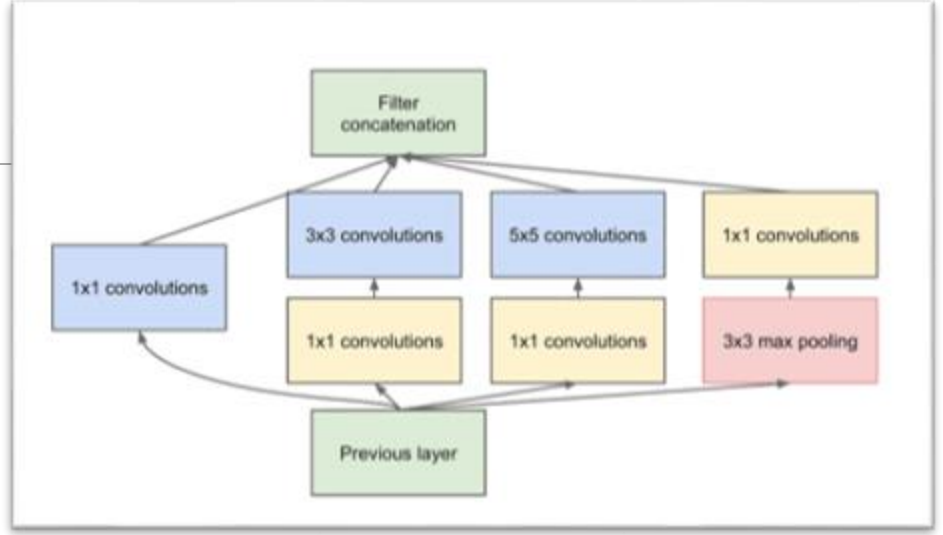
Deep architecture

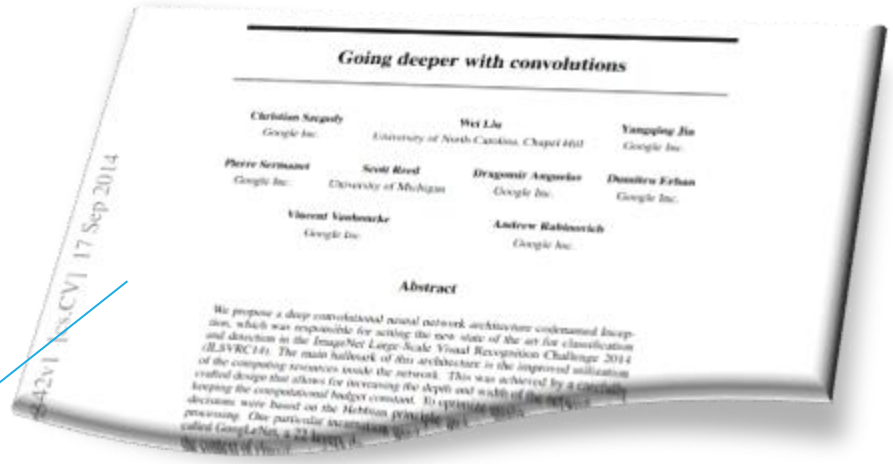
- 22 layers optimized for fewer parameters
- Global Average Pooling instead of large fully connected layers

Intermediary classifiers to improve learning

- 2 auxiliary classifiers attached during training
- Combat the vanishing gradient problem

More efficient than VGG




Going deeper with convolutions

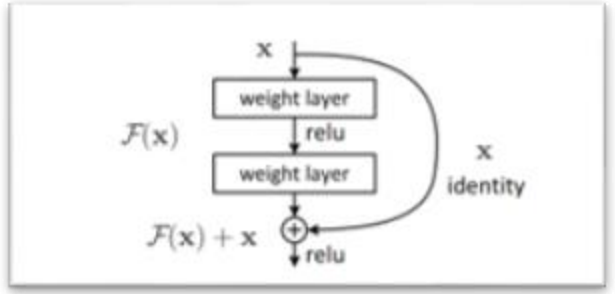
Abstract

https://arxiv.org/pdf/1409.4842

ResNet

Residual blocks

- Input is added directly to output to learn residual functions
- Skip connections allow gradients to flow more efficiently

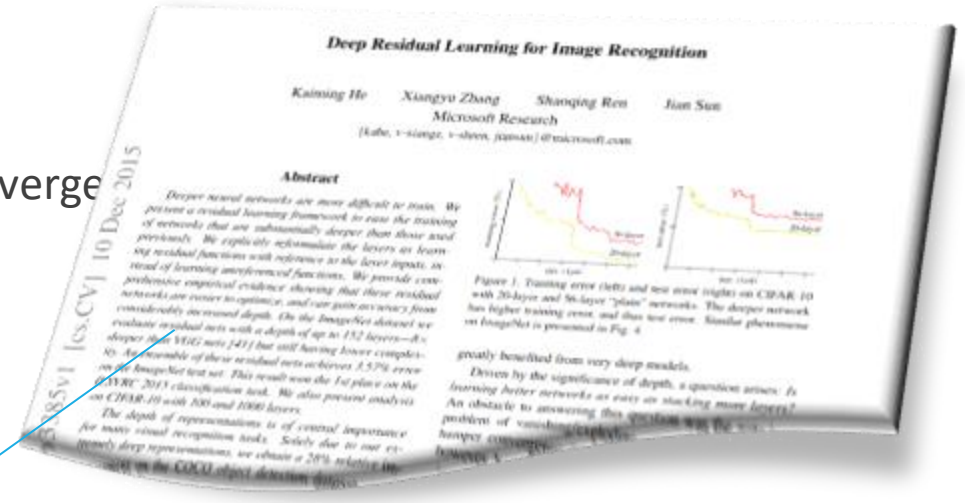


Scalable architecture

- Variants include 34, 50, 101, or 152 layers
- Deeper versions use bottleneck blocks for parameter efficiency

Training stability

- Batch normalization and ReLU after convolutions to improve converge



https://arxiv.org/pdf/1512.03385

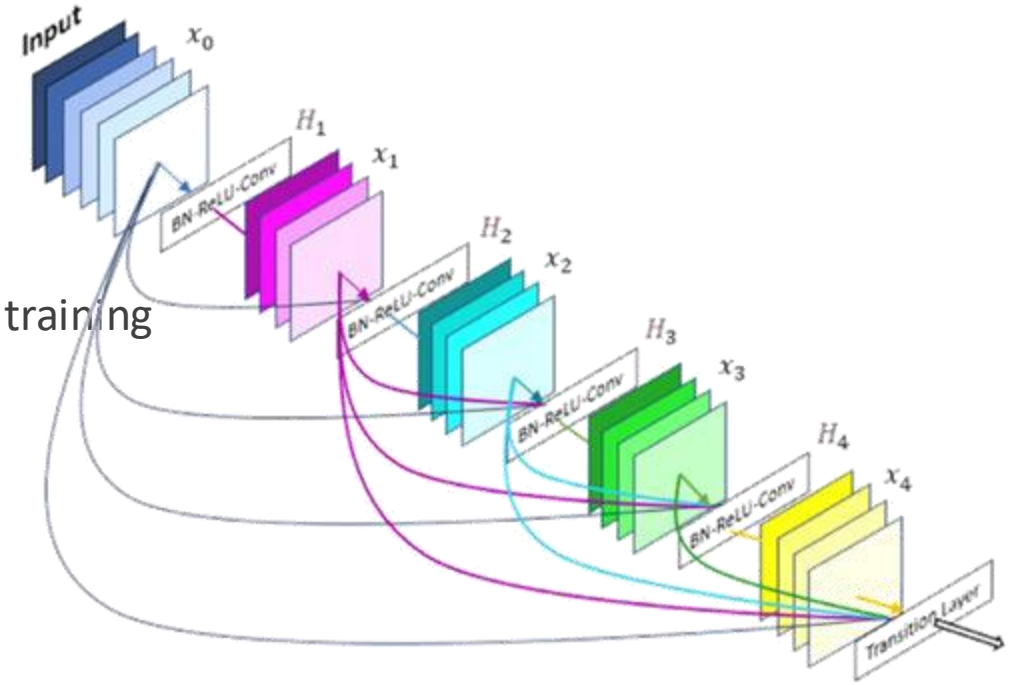
DenseNet

Dense blocks
- Multiple convolutional layers with batch normalization and non-linear activation (ReLU)
- Every layer receives all preceding layers as input

Transition layers
- Connecting dense blocks
- Batch normalization, 1x1 convolution and average pooling

Alleviated vanishing gradient problem
- Direct paths from every layer ensure strong gradient signals in training

Dense connectivity
- Extensive feature reuse
- High computational complexity
- Variants include 121, 169, 201, or 264 layers





Lab Practice 4

Applying transfer learning

Enrique Dominguez

DEEP LEARNING